

BAB 2: INTEGRASI KERANGKA KERJA STEM DAN PEMIKIRAN KOMPUTASIONAL

PENDAHULUAN

Pendidikan matematika secara tradisional seringkali dipersepsikan sebagai sebuah disiplin yang berdiri sendiri, terisolasi dari mata pelajaran lain dan dari konteks dunia nyata. Siswa belajar rumus dan prosedur dalam sebuah ruang hampa konseptual, yang seringkali menyebabkan munculnya pertanyaan klasik: “kapan saya akan menggunakan ini dalam kehidupan nyata?”. Keterpisahan ini tidak hanya mengurangi motivasi belajar, tetapi juga gagal membekali siswa dengan kemampuan untuk menerapkan pengetahuan matematis mereka dalam memecahkan masalah-masalah multidisiplin yang akan mereka hadapi di masa depan. Diperlukan sebuah pergeseran fundamental dari pendekatan yang terkotak-kotak menuju sebuah paradigma yang lebih terintegrasi dan kontekstual.

Kerangka kerja *Science, Technology, Engineering, and Mathematics* (STEM) muncul sebagai respons langsung terhadap tantangan ini. STEM bukanlah sekadar akronim yang menggabungkan empat mata pelajaran, melainkan sebuah filosofi pendidikan yang menekankan pembelajaran berbasis masalah dan proyek dalam konteks dunia nyata (Bybee, 2013). Dalam pendekatan STEM terintegrasi, matematika tidak lagi diajarkan sebagai tujuan akhir, tetapi sebagai bahasa universal dan alat analisis esensial yang digunakan untuk memahami fenomena ilmiah, merancang solusi teknologi, dan mengoptimalkan proses rekayasa. Integrasi ini mengubah pembelajaran matematika dari aktivitas abstrak menjadi sebuah proses yang hidup dan relevan.

Sejalan dengan integrasi STEM, pemikiran komputasional (*Computational Thinking* - CT) telah diidentifikasi sebagai sebuah kompetensi fundamental yang melintasi semua disiplin ilmu, termasuk matematika (Grover & Pea, 2013). CT bukanlah tentang berpikir seperti mesin, melainkan seperangkat keterampilan pemecahan masalah yang digunakan oleh ilmuwan komputer untuk merancang sistem yang kompleks. Keterampilan ini, seperti dekomposisi, pengenalan pola, abstraksi, dan desain algoritma, memiliki kesamaan yang mendalam dengan proses penalaran matematis. Mengintegrasikan CT ke dalam kelas matematika dapat memperkuat kemampuan siswa untuk mendekati masalah secara sistematis dan logis.

Sinergi antara STEM dan CT menawarkan sebuah model pendidikan matematika yang sangat kuat untuk abad ke-21. Ketika siswa dihadapkan pada masalah STEM yang kompleks, misalnya, merancang model untuk memprediksi penyebaran penyakit atau mengoptimalkan efisiensi panel surya, mereka

tidak hanya perlu menerapkan konsep-konsep matematika, tetapi juga perlu menggunakan keterampilan CT untuk mengelola kompleksitas masalah tersebut. Mereka harus memecah masalah menjadi bagian-bagian yang lebih kecil, mengidentifikasi pola dalam data, mengabstraksi detail yang tidak perlu untuk membuat model, dan merancang serangkaian langkah logis (algoritma) untuk mencapai solusi.

Teknologi digital, khususnya platform komputasi visual seperti Scratch dan Python, memainkan peran krusial sebagai katalisator untuk integrasi STEM dan CT. Lingkungan pemrograman ini memungkinkan siswa untuk membangun simulasi interaktif dari fenomena ilmiah, memvisualisasikan konsep-konsep geometris yang kompleks, dan menguji algoritma mereka secara langsung. Dengan membuat model komputasional, siswa tidak hanya belajar matematika dan sains secara pasif; mereka secara aktif membangun representasi pengetahuan mereka, sebuah proses yang terbukti dapat menghasilkan pemahaman konseptual yang lebih dalam dan bertahan lama (Brennan & Resnick, 2012).

Tantangan utama dalam implementasi integrasi ini adalah perancangan kurikulum dan pengembangan profesional bagi para guru. Diperlukan sebuah pergeseran dari kurikulum yang didorong oleh konten menuju kurikulum yang didorong oleh masalah (*problem-driven*). Guru perlu dibekali dengan strategi pedagogis untuk memfasilitasi pembelajaran berbasis proyek interdisipliner dan untuk menilai kompetensi-kompetensi seperti CT yang bersifat proses, bukan hanya jawaban akhir. Ini menuntut perubahan peran guru dari penyampai informasi menjadi seorang arsitek pengalaman belajar.

Bab ini akan mengupas secara mendalam fondasi teoretis dan strategi praktis untuk mengintegrasikan kerangka kerja STEM dan pemikiran komputasional ke dalam pendidikan matematika. Pembahasan akan mencakup karakteristik pembelajaran STEM yang terintegrasi, komponen-komponen utama CT, dan pemanfaatan platform komputasi visual sebagai jembatan antara keduanya. Tujuan dari bab ini adalah untuk memberikan landasan yang kokoh bagi para pendidik untuk merancang pengalaman belajar matematika yang tidak hanya relevan secara kontekstual, tetapi juga membekali siswa dengan keterampilan berpikir tingkat tinggi yang esensial.

Dengan demikian, bab ini akan menunjukkan bahwa matematika, ketika diintegrasikan dengan STEM dan diperkuat dengan CT, dapat bertransformasi menjadi sebuah disiplin yang dinamis, kreatif, dan sangat memberdayakan. Siswa tidak hanya belajar "tentang" matematika, tetapi mereka "melakukan" matematika dalam konteks yang bermakna, mempersiapkan mereka untuk menjadi pemecah masalah yang inovatif dan adaptif dalam menghadapi tantangan-tantangan kompleks di masa depan, baik dalam karir mereka maupun dalam kehidupan sebagai warga negara yang terinformasi.

2.1. Fondasi STEM dalam Pendidikan Matematika

Pendekatan STEM dalam pendidikan seringkali disalahpahami sebagai sekadar pengajaran empat mata pelajaran secara terpisah dalam satu payung kurikulum. Namun, esensi sejati dari pendidikan STEM terletak pada integrasi, yaitu sebuah pendekatan interdisipliner dan transdisipliner di mana konsep-konsep akademis yang kaku dipadukan dengan pelajaran dunia nyata (Sanders, 2009). Dalam model ini, siswa menerapkan pengetahuan dari sains, teknologi, rekayasa, dan matematika untuk memecahkan masalah-masalah yang relevan dan kontekstual. Matematika dalam kerangka ini berhenti menjadi entitas yang terisolasi dan bertransformasi menjadi bahasa fundamental yang memungkinkan kuantifikasi, pemodelan, dan analisis di seluruh domain STEM lainnya.

Sebuah kerangka kerja konseptual yang berpengaruh untuk memahami integrasi ini diajukan oleh Kelley dan Knowles (2016), yang mendefinisikan pendidikan STEM terintegrasi sebagai pendekatan yang mengeksplorasi pengajaran dan pembelajaran di antara dua atau lebih bidang studi STEM, atau antara satu bidang studi STEM dan satu atau lebih disiplin sekolah lainnya. Fondasi dari pendekatan ini adalah pembelajaran yang berpusat pada masalah (*problem-centered learning*). Alih-alih dimulai dengan teori matematika abstrak, pembelajaran dimulai dengan sebuah tantangan atau masalah dunia nyata yang otentik, seperti merancang sistem penyaringan air yang efisien atau memodelkan dampak perubahan iklim terhadap ekosistem lokal.

Masalah-masalah otentik ini secara inheren bersifat interdisipliner. Untuk menyelesaikan tantangan sistem penyaringan air, misalnya, siswa perlu memahami prinsip-prinsip ilmiah tentang permeabilitas dan kontaminasi (Sains). Mereka mungkin perlu menggunakan sensor digital dan perangkat lunak *spreadsheet* untuk mengumpulkan dan menganalisis data kualitas air (Teknologi). Proses merancang, membangun, dan menguji prototipe filter adalah inti dari praktik rekayasa (Rekayasa). Dan di sepanjang proses tersebut, mereka harus menggunakan konsep-konsep matematika seperti laju alir, rasio, persentase, dan analisis statistik untuk mengukur dan mengoptimalkan kinerja desain mereka (Matematika).

Dalam konteks ini, pembelajaran matematika menjadi terdorong oleh kebutuhan (*need-driven*). Siswa mempelajari konsep-konsep matematis bukan karena konsep tersebut muncul di bab berikutnya dalam buku teks, tetapi karena mereka membutuhkannya untuk membuat kemajuan dalam proyek mereka. Motivasi intrinsik ini secara signifikan meningkatkan keterlibatan dan retensi pengetahuan. Peran matematika menjadi jelas sebagai alat yang kuat untuk pemodelan dan pengambilan keputusan, bukan sekadar serangkaian prosedur yang harus dihafal.

Implementasi pendidikan STEM yang efektif menuntut perubahan signifikan dalam pedagogi. Pembelajaran berbasis proyek (*Project-Based Learning* - PBL) dan pembelajaran berbasis masalah (*Problem-Based Learning* - PjBL) menjadi strategi instruksional utama. Guru bertindak sebagai fasilitator, membimbing siswa melalui siklus penyelidikan, eksperimen, dan iterasi desain. Penekanan bergeser dari penguasaan konten yang terfragmentasi ke pengembangan kompetensi holistik, termasuk pemikiran kritis, kolaborasi, komunikasi, dan kreativitas.

Keterkaitan antar disiplin ilmu menjadi fokus eksplisit dalam analisis. Guru harus secara sengaja membantu siswa melihat bagaimana, misalnya, model persamaan diferensial (Matematika) digunakan untuk menggambarkan pertumbuhan populasi dalam biologi (Sains), atau bagaimana prinsip-prinsip geometri (Matematika) sangat penting dalam desain struktur yang stabil dalam rekayasa sipil (Rekayasa). Membuat koneksi-koneksi ini secara eksplisit membantu siswa membangun peta kognitif yang lebih kaya dan lebih terhubung tentang bagaimana pengetahuan bekerja di dunia nyata.

Perancangan kurikulum matematika bernuansa STEM memerlukan pemikiran ulang tentang urutan dan penekanan topik. Topik-topik seperti statistika, probabilitas, dan pemodelan matematika, yang seringkali ditempatkan di akhir kurikulum tradisional, menjadi sangat sentral dalam pendekatan STEM karena relevansinya yang tinggi dalam analisis data dan pemodelan fenomena dunia nyata. Kurikulum harus fleksibel untuk memungkinkan eksplorasi mendalam terhadap proyek-proyek yang mungkin memakan waktu beberapa minggu, mengintegrasikan berbagai standar kompetensi dalam satu pengalaman belajar yang koheren.

Pada akhirnya, fondasi STEM dalam pendidikan matematika adalah tentang kontekstualisasi dan relevansi. Ini adalah tentang mengubah matematika dari sebuah "apa" (kumpulan fakta dan prosedur) menjadi sebuah "bagaimana" (sebuah cara berpikir dan seperangkat alat untuk memahami dan membentuk dunia). Dengan menempatkan matematika dalam matriks pemecahan masalah STEM yang kaya, pendidik dapat membuka potensi penuh dari disiplin ini untuk menginspirasi generasi baru para inovator, pemikir kritis, dan warga negara yang berpengetahuan luas.

Analogi/Contoh Kasus: Mengintegrasikan matematika ke dalam STEM dapat dianalogikan dengan peran seorang ahli nutrisi dalam tim atletik profesional. Di masa lalu, ahli nutrisi mungkin bekerja secara terpisah, memberikan rencana makan standar (seperti matematika yang diajarkan terpisah). Namun, dalam pendekatan terintegrasi modern, ahli nutrisi bekerja sama erat dengan pelatih kekuatan (Sains), analis data kinerja (Teknologi), dan fisioterapis (Rekayasa tubuh). Rekomendasi nutrisi (Matematika, dalam bentuk kalori, makronutrien, waktu makan) tidak lagi bersifat umum, melainkan disesuaikan secara dinamis berdasarkan data latihan, tujuan kekuatan, dan kebutuhan pemulihan cedera.

Di sini, nutrisi (matematika) menjadi alat yang esensial dan relevan secara kontekstual untuk mencapai tujuan tim yang lebih besar, yaitu kinerja atletik puncak.

2.1.1. Karakteristik pembelajaran berbasis STEM yang terintegrasi

Pembelajaran berbasis STEM yang terintegrasi secara otentik memiliki beberapa karakteristik yang membedakannya dari pengajaran mata pelajaran secara terpisah. Karakteristik utamanya adalah fokus pada masalah dunia nyata sebagai titik awal dan pusat pembelajaran. Masalah ini harus relevan dengan kehidupan siswa dan cukup kompleks untuk menuntut penerapan pengetahuan dan keterampilan dari setidaknya dua disiplin STEM (Kelley & Knowles, 2016). Pembelajaran menjadi sebuah proses penyelidikan di mana siswa didorong untuk mengajukan pertanyaan, melakukan penelitian, dan mengembangkan solusi, bukan sekadar menyerap informasi.

Karakteristik penting lainnya adalah penekanan pada proses desain rekayasa (*engineering design process*). Siklus yang berulang ini, meliputi identifikasi masalah, curah pendapat solusi, perancangan, pembuatan prototipe, pengujian, dan perbaikan, memberikan struktur untuk pembelajaran berbasis proyek. Matematika dan sains tidak diajarkan sebagai teori murni, tetapi sebagai pengetahuan yang diperlukan untuk menginformasikan dan mengevaluasi setiap tahap dalam siklus desain tersebut (Bybee, 2010). Proses ini secara alami mendorong pengembangan keterampilan berpikir tingkat tinggi seperti analisis, evaluasi, dan kreasi.

2.1.2. Strategi perancangan kurikulum matematika bernuansa STEM

Merancang kurikulum matematika bernuansa STEM memerlukan pergeseran dari pendekatan berbasis topik (*topic-based*) ke pendekatan berbasis tema atau masalah (*theme/problem-based*). Salah satu strategi efektif adalah "bundling" atau mengelompokkan standar kompetensi dari berbagai mata pelajaran di sekitar sebuah proyek atau tema sentral. Misalnya, sebuah proyek tentang "Desain Komunitas Berkelanjutan" dapat mengintegrasikan standar kompetensi dari geometri dan aljabar (desain tata letak, optimalisasi ruang), fisika (efisiensi energi), biologi (ekosistem lokal), dan ilmu sosial.

Strategi lainnya adalah "backwards design", di mana perancangan dimulai dari tujuan akhir, yaitu produk atau solusi yang akan dihasilkan siswa dalam sebuah proyek. Pendidik kemudian bekerja mundur untuk mengidentifikasi pengetahuan dan keterampilan matematika, sains, dan teknis apa yang perlu dipelajari siswa untuk berhasil menyelesaikan proyek tersebut. Pendekatan ini memastikan bahwa semua pembelajaran konten secara langsung relevan dengan tujuan akhir dan membantu menjaga fokus serta motivasi siswa sepanjang proyek berlangsung. Fleksibilitas kurikulum juga menjadi kunci, memberikan ruang bagi guru dan siswa untuk mengejar jalur penyelidikan yang tidak terduga yang mungkin muncul selama proses proyek.

2.1.3. Analisis keterkaitan disiplin ilmu dalam kerangka STEM

Analisis keterkaitan disiplin ilmu dalam STEM menunjukkan bahwa matematika memainkan peran sebagai bahasa dan fondasi analitis. Sains menyediakan pemahaman konseptual tentang dunia alam, tetapi matematika memberikan alat untuk mengkuantifikasi, memodelkan, dan memprediksi fenomena ilmiah tersebut. Misalnya, hukum gerak Newton (Sains) dijelaskan secara presisi melalui persamaan diferensial (Matematika). Rekayasa berfokus pada perancangan solusi untuk masalah manusia, tetapi proses desain tersebut sangat bergantung pada pemodelan matematis untuk mengoptimalkan kinerja dan memastikan keamanan.

Teknologi, dalam kerangka STEM, seringkali berfungsi sebagai jembatan yang menghubungkan disiplin-disiplin lain. Perangkat lunak simulasi (Teknologi) memungkinkan para ilmuwan dan insinyur untuk menguji model matematis mereka dalam lingkungan virtual sebelum menerapkannya di dunia nyata. Sebaliknya, kemajuan dalam matematika, seperti pengembangan algoritma baru, seringkali mendorong inovasi teknologi. Dengan demikian, keempat disiplin ini berada dalam hubungan simbiosis: kemajuan di satu bidang seringkali bergantung pada dan memicu kemajuan di bidang lainnya, dengan matematika sebagai benang merah yang menyatukan semuanya (Sanders, 2009).

2.2. *Computational Thinking* (CT) dalam Matematika

Pemikiran komputasional (*Computational Thinking* - CT) adalah sebuah pendekatan untuk pemecahan masalah yang melibatkan serangkaian proses mental dan praktik yang digunakan oleh para ilmuwan komputer, namun memiliki penerapan yang jauh lebih luas (Grover & Pea, 2013). CT bukanlah tentang membuat manusia berpikir seperti komputer; sebaliknya, ini adalah tentang memberdayakan manusia untuk merumuskan masalah dan solusinya dalam bentuk yang dapat dieksekusi secara efektif oleh agen pemroses informasi, baik itu komputer maupun manusia lain. Dalam pendidikan matematika, integrasi CT menawarkan sebuah kerangka kerja yang kuat untuk memperdalam penalaran prosedural dan konseptual.

Fondasi dari CT terdiri dari empat komponen utama yang saling terkait. Pertama adalah **dekomposisi**, yaitu kemampuan untuk memecah masalah yang besar dan kompleks menjadi bagian-bagian yang lebih kecil dan lebih mudah dikelola. Proses ini sangat familiar dalam matematika, misalnya saat memecah soal cerita yang rumit menjadi beberapa langkah perhitungan yang lebih sederhana. Secara eksplisit mengajarkan dekomposisi sebagai strategi membantu siswa mengelola beban kognitif dan mendekati masalah yang tampak menakutkan dengan lebih percaya diri.

Komponen kedua adalah **pengenalan pola**. Ini adalah kemampuan untuk mengidentifikasi kesamaan, tren, atau keteraturan dalam data atau di antara masalah-masalah yang berbeda. Dalam matematika,

pengenalan pola adalah inti dari generalisasi. Ketika siswa menyadari bahwa proses untuk menemukan luas persegi panjang, segitiga, dan trapesium memiliki struktur yang serupa (melibatkan perkalian dimensi dasar), mereka sedang melakukan pengenalan pola yang mengarah pada pemahaman konsep yang lebih dalam tentang pengukuran luas.

Komponen ketiga adalah **abstraksi**, yaitu proses memfokuskan pada informasi yang penting sambil mengabaikan detail yang tidak relevan. Pembuatan model matematis adalah bentuk abstraksi yang murni. Ketika seorang siswa membuat persamaan untuk memodelkan gerak parabola sebuah bola, mereka mengabstraksi detail-detail seperti warna bola atau hambatan udara (pada tingkat awal) untuk fokus pada variabel-variabel kunci seperti kecepatan awal, sudut, dan gravitasi. Abstraksi adalah keterampilan krusial yang memungkinkan generalisasi dan penerapan solusi untuk berbagai masalah.

Komponen keempat dan yang paling sentral adalah **desain algoritma**, yaitu pengembangan serangkaian instruksi atau aturan langkah-demi-langkah untuk menyelesaikan sebuah masalah. Banyak prosedur matematika, mulai dari algoritma pembagian panjang hingga metode penyelesaian sistem persamaan linear, pada dasarnya adalah algoritma. Mengajarkan siswa untuk tidak hanya mengikuti algoritma, tetapi juga merancang, menganalisis, dan membandingkan algoritma yang berbeda (misalnya, membandingkan efisiensi dua metode pengurutan data) dapat secara signifikan meningkatkan pemahaman mereka tentang proses matematis.

Integrasi CT ke dalam matematika tidak memerlukan penambahan materi baru yang besar ke dalam kurikulum. Sebaliknya, ini lebih tentang membuat proses berpikir yang sudah implisit dalam matematika menjadi lebih eksplisit. Guru dapat menggunakan bahasa CT untuk membimbing diskusi di kelas. Misalnya, saat memulai masalah baru, guru dapat bertanya, "Bagaimana kita bisa memecah (*decompose*) masalah ini?" atau setelah menyelesaikan beberapa contoh, "Pola apa yang kita lihat di sini?".

Penggunaan pemrograman dasar, bahkan dalam lingkungan visual seperti Scratch, dapat menjadi cara yang sangat efektif untuk membuat konsep CT menjadi konkret. Ketika siswa menulis sebuah program untuk menggambar sebuah poligon beraturan, mereka harus secara eksplisit mendefinisikan algoritma (misalnya, "maju sekian langkah, putar sekian derajat, ulangi N kali"). Proses *debugging* atau mencari kesalahan dalam program mereka adalah latihan yang sangat baik dalam pemikiran logis dan sistematis, keterampilan yang dapat ditransfer secara langsung ke pemecahan masalah matematika.

Lebih jauh, CT mendorong pola pikir yang menghargai ketekunan dan toleransi terhadap ambiguitas. Dalam pemrograman, jarang sekali sebuah program yang kompleks bekerja dengan sempurna pada percobaan pertama. Siswa belajar untuk mendekati kesalahan bukan sebagai kegagalan, tetapi sebagai

data yang berharga untuk perbaikan. Pola pikir iteratif dan berkembang (*iterative and growth mindset*) ini sangat bermanfaat dalam menghadapi masalah-masalah matematika yang menantang, di mana solusi seringkali tidak langsung terlihat.

Pada akhirnya, integrasi CT memperkaya pendidikan matematika dengan menyediakan sebuah meta-kerangka untuk pemecahan masalah. Ia membekali siswa dengan seperangkat alat konseptual yang dapat diterapkan pada berbagai macam tantangan, baik di dalam maupun di luar kelas matematika. Dengan memadukan ketepatan penalaran matematis dengan kekuatan pemecahan masalah yang sistematis dari CT, pendidik dapat mempersiapkan siswa untuk menjadi pemikir yang lebih fleksibel, tangguh, dan kuat dalam dunia yang semakin didorong oleh komputasi.

Analogi/Contoh Kasus: Mengintegrasikan CT dalam matematika dapat dianalogikan dengan proses belajar memasak resep yang kompleks. Seorang koki pemula mungkin hanya mengikuti resep langkah demi langkah secara mekanis (mengikuti algoritma). Namun, seorang koki dengan *computational thinking* akan mendekatinya secara berbeda. Ia akan terlebih dahulu **mendekomposisi** resep menjadi komponen-komponen utama (misalnya, persiapan sayuran, pembuatan saus, memasak protein). Ia akan mencari **pola** dari resep-resep lain yang pernah ia masak (misalnya, "ah, ini teknik dasar menumis yang sama"). Ia akan melakukan **abstraksi** dengan fokus pada rasio bahan-bahan kunci dan mengabaikan saran merek tertentu. Akhirnya, ia tidak hanya mengikuti resep, tetapi ia memahami **algoritma** di baliknya, yang memungkinkannya untuk memodifikasi, memperbaiki, atau bahkan menciptakan resep baru.

2.2.1. Komponen utama Computational Thinking dalam pemecahan masalah

Pemikiran komputasional dapat dipecah menjadi empat komponen fundamental yang bekerja secara sinergis dalam proses pemecahan masalah. Komponen pertama, **dekomposisi**, adalah praktik memecah masalah yang kompleks dan besar menjadi sub-masalah yang lebih kecil dan lebih mudah dikelola. Dalam konteks matematika, saat dihadapkan pada soal optimisasi multi-langkah, seorang siswa yang menerapkan dekomposisi akan terlebih dahulu mengidentifikasi variabel-variabel, kemudian merumuskan fungsi tujuan, dan akhirnya menentukan batasan-batasan sebagai langkah-langkah terpisah sebelum mencoba menyelesaikannya secara keseluruhan.

Komponen kedua, **pengenalan pola**, adalah kemampuan untuk mencari kesamaan atau tren. Setelah memecahkan beberapa masalah, siswa mungkin mengenali bahwa masalah-masalah yang tampaknya berbeda sebenarnya memiliki struktur matematis yang sama (misalnya, berbagai skenario yang dapat dimodelkan dengan pertumbuhan eksponensial). Kemampuan ini memungkinkan transfer pengetahuan

dari satu konteks ke konteks lain. **Abstraksi**, komponen ketiga, melibatkan penyaringan detail-detail yang tidak perlu untuk fokus pada struktur inti dari masalah. Ini adalah jantung dari pemodelan matematis, di mana situasi dunia nyata yang berantakan disederhanakan menjadi sistem persamaan atau representasi geometris yang dapat dianalisis. Terakhir, **desain algoritma** adalah formulasi serangkaian langkah yang jelas dan terurut untuk menyelesaikan masalah atau sub-masalah. Ini mengubah solusi konseptual menjadi sebuah proses yang dapat dieksekusi dan diulang (Grover & Pea, 2018).

2.2.2. Integrasi algoritma dalam proses berpikir matematis

Meskipun matematika penuh dengan algoritma (misalnya, algoritma Euklides untuk menemukan faktor persekutuan terbesar), pengajaran tradisional seringkali menyajikannya sebagai resep yang harus dihafal. Pendekatan yang diperkaya dengan CT mendorong siswa untuk tidak hanya menggunakan algoritma, tetapi juga untuk memahami, memodifikasi, dan bahkan menciptakannya. Ini berarti menganalisis mengapa sebuah algoritma bekerja dan dalam kondisi apa ia efisien.

Integrasi ini dapat dicapai dengan mengajukan pertanyaan-pertanyaan yang mendorong pemikiran algoritmik. Misalnya, setelah mengajarkan metode faktorisasi, guru dapat bertanya, "Bisakah kita menuliskan serangkaian instruksi yang sangat jelas yang dapat diikuti oleh seseorang yang belum pernah melihat masalah ini untuk menyelesaikannya?". Aktivitas seperti membandingkan efisiensi dua metode berbeda untuk menyelesaikan sistem persamaan (misalnya, substitusi vs. eliminasi) juga merupakan bentuk analisis algoritmik. Tujuannya adalah untuk menggeser persepsi siswa dari algoritma sebagai prosedur yang kaku menjadi alat pemecahan masalah yang fleksibel dan dapat dirancang.

2.2.3. Pengembangan pola pikir logis melalui pemrograman dasar

Pemrograman dasar, terutama di lingkungan visual seperti Scratch atau Blockly, menyediakan arena latihan yang sangat baik untuk mengembangkan pola pikir logis yang krusial dalam matematika. Komputer menuntut presisi absolut; instruksi yang ambigu atau tidak logis akan menghasilkan kesalahan atau perilaku yang tidak terduga. Proses ini memaksa siswa untuk berpikir secara sangat jernih dan sistematis tentang urutan langkah-langkah dan hubungan sebab-akibat.

Ketika siswa mencoba membuat program untuk menggambar sebuah persegi, mereka dengan cepat belajar bahwa instruksi "maju, belok, maju, belok" tidak cukup. Mereka harus secara spesifik menentukan "maju berapa langkah?" dan "belok berapa derajat?". Proses *debugging*, mencari dan memperbaiki kesalahan dalam kode mereka, adalah latihan pemecahan masalah yang sangat berharga. Ini melatih siswa untuk mengisolasi variabel, menguji hipotesis, dan berpikir secara logis tentang di mana letak kesalahan dalam alur penalaran mereka, sebuah keterampilan yang sangat dapat ditransfer ke proses memeriksa pekerjaan dalam soal-soal matematika yang kompleks (Resnick, 2017).

2.3. Platform Komputasi Visual untuk Matematika

Transisi dari konsep-konsep abstrak pemikiran komputasional ke praktik nyata di dalam kelas difasilitasi secara signifikan oleh platform komputasi visual. Platform-platform ini, seperti Scratch yang dikembangkan oleh MIT Media Lab atau lingkungan berbasis blok lainnya, secara sengaja dirancang untuk menurunkan ambang batas teknis dalam pemrograman. Alih-alih mengharuskan siswa untuk menghafal sintaksis yang rumit dari bahasa pemrograman berbasis teks, lingkungan visual memungkinkan mereka untuk menyusun program dengan menyeret dan melepaskan blok-blok kode berwarna yang saling terkait seperti potongan puzzle. Pendekatan ini memungkinkan siswa, bahkan di tingkat sekolah dasar, untuk segera mulai bereksperiensi dan bereksperimen dengan konsep-konsep komputasional yang kuat.

Dalam konteks pendidikan matematika, platform ini berfungsi sebagai laboratorium virtual atau "bak pasir matematis" (*math sandbox*). Di lingkungan ini, konsep-konsep matematika yang seringkali tampak abstrak dan statis, seperti variabel, operator, dan loop, menjadi entitas yang nyata dan dapat dimanipulasi. Sebuah variabel bukan lagi sekadar huruf 'x' di papan tulis; ia menjadi sebuah "wadah" digital yang nilainya dapat dilihat dan diubah secara dinamis, dan perubahan tersebut secara instan memengaruhi perilaku sebuah karakter atau objek di layar. Pengalaman yang diwujudkan (*embodied experience*) ini dapat membantu membangun fondasi intuisi yang kuat sebelum siswa beralih ke notasi matematika yang lebih formal.

Implementasi skrip sederhana dalam platform visual ini dapat digunakan untuk memodelkan dan mensimulasikan berbagai macam konsep matematika. Dalam geometri, misalnya, siswa dapat dengan mudah menulis program untuk memerintahkan karakter menggambar bentuk-bentuk geometris. Proses ini mengubah pembelajaran geometri dari sekadar pengenalan bentuk menjadi sebuah penyelidikan tentang sifat-sifat yang mendasari bentuk tersebut. Untuk menggambar sebuah persegi, siswa harus memahami hubungan antara panjang sisi yang sama dan sudut 90 derajat. Untuk menggambar poligon beraturan, mereka harus menemukan dan menerapkan hubungan antara jumlah sisi dan sudut eksteriornya, sebuah penemuan konsep yang jauh lebih berkesan daripada sekadar menghafal rumus.

Efektivitas platform komputasi visual terletak pada kemampuannya untuk mendukung siklus belajar yang digagas oleh Seymour Papert, yaitu konstruksionisme. Teori ini berpendapat bahwa pembelajaran terjadi paling efektif ketika siswa secara aktif terlibat dalam membangun artefak yang bermakna secara pribadi, baik itu sebuah cerita interaktif, permainan, atau simulasi. Dalam proses membangun artefak ini, siswa secara alami akan menghadapi dan bergulat dengan ide-ide matematis dan komputasional yang kuat. Mereka tidak belajar "tentang" variabel; mereka "menggunakan" variabel untuk membuat papan skor dalam permainan yang mereka rancang (Resnick, 2017).

Selain itu, sifat terbuka (*open-ended*) dari platform ini mendorong kreativitas dan ekspresi pribadi. Siswa dapat mengerjakan proyek-proyek yang selaras dengan minat mereka, baik itu seni, musik, atau olahraga, sambil tetap terlibat dengan konsep-konsep matematika dan CT inti. Motivasi yang didorong oleh minat pribadi ini terbukti menjadi faktor kunci dalam menumbuhkan ketekunan dalam menghadapi tantangan (Brennan & Resnick, 2013). Komunitas daring yang seringkali terintegrasi dengan platform ini juga memungkinkan siswa untuk berbagi karya mereka, mendapatkan inspirasi, dan belajar dari satu sama lain, menumbuhkan lingkungan belajar sosial yang melampaui batas-batas ruang kelas.

Tentu saja, efektivitas pedagogis dari alat-alat ini tidak datang secara otomatis. Penggunaannya harus dibingkai dengan tujuan pembelajaran yang jelas dan difasilitasi oleh guru yang terampil. Guru perlu merancang tantangan-tantangan yang mendorong siswa untuk menjelajahi konsep-konsep matematika tertentu dan mengajukan pertanyaan-pertanyaan yang memprovokasi refleksi tentang proses berpikir mereka. Evaluasi tidak boleh hanya berfokus pada produk akhir (apakah programnya bekerja?), tetapi juga pada proses: Bagaimana siswa mendekati masalah? Strategi *debugging* apa yang mereka gunakan? Konsep komputasional apa yang mereka terapkan?

Bagi siswa yang lebih mahir atau berada di tingkat yang lebih tinggi, transisi ke bahasa pemrograman berbasis teks seperti Python menjadi langkah berikutnya yang logis. Python, dengan sintaksisnya yang relatif bersih dan perpustakaan (*libraries*) yang kuat untuk komputasi ilmiah dan visualisasi data (seperti NumPy dan Matplotlib), membuka kemungkinan yang lebih luas untuk pemodelan STEM yang canggih. Namun, pengalaman awal dengan platform visual memberikan fondasi konseptual yang tak ternilai, membuat transisi ke pemrograman berbasis teks menjadi tidak terlalu menakutkan dan lebih intuitif.

Secara keseluruhan, platform komputasi visual merepresentasikan sebuah jembatan yang kuat antara dunia abstrak matematika dan pemikiran komputasional dengan dunia konkret ekspresi dan kreasi siswa. Mereka menyediakan lingkungan dengan "dinding rendah dan langit-langit tinggi" (*low floor, high ceiling*), artinya mudah untuk memulai tetapi menawarkan ruang yang hampir tak terbatas untuk pertumbuhan dan kompleksitas. Dengan mengintegrasikan alat-alat ini secara bijaksana, pendidik dapat menciptakan pengalaman belajar matematika yang lebih menarik, personal, dan secara konseptual kaya.

Analogi/Contoh Kasus: Penggunaan platform komputasi visual dalam matematika dapat dianalogikan dengan menggunakan balok LEGO untuk belajar tentang arsitektur. Daripada hanya melihat gambar-gambar bangunan dan membaca tentang prinsip-prinsip struktural (pembelajaran pasif), anak-anak diberikan satu set balok LEGO (platform visual). Mereka dapat secara langsung membangun, menguji ide, dan melihat secara fisik mengapa sebuah struktur runtuh atau mengapa yang lain kokoh. Mereka

belajar tentang konsep-konsep seperti fondasi, distribusi beban, dan penopang (konsep matematika/fisika) bukan karena disuruh menghafal, tetapi karena mereka membutuhkannya untuk membuat menara mereka lebih tinggi. Prosesnya menyenangkan, iteratif, dan kesalahan (menara runtuh) adalah bagian penting dari pembelajaran, bukan kegagalan.

2.3.1. Pengenalan lingkungan pemrograman visual (Scratch/Python)

Lingkungan pemrograman visual seperti Scratch dirancang dengan prinsip "lantai rendah, langit-langit tinggi, dan dinding lebar". "Lantai rendah" berarti pemula dapat dengan mudah memulai; antarmuka seret-dan-lepas menghilangkan kebutuhan untuk mengkhawatirkan kesalahan sintaksis, memungkinkan siswa untuk fokus langsung pada logika program mereka. "Langit-langit tinggi" berarti platform ini cukup kuat untuk membuat proyek-proyek yang sangat kompleks seiring dengan berkembangnya keterampilan siswa. "Dinding lebar" berarti platform ini mendukung berbagai jenis proyek yang berbeda, animasi, permainan, simulasi, sehingga dapat menarik minat yang beragam.

Python, di sisi lain, adalah bahasa pemrograman berbasis teks serbaguna yang banyak digunakan dalam industri dan akademisi. Meskipun kurva belajarnya lebih curam daripada Scratch, sintaksisnya yang bersih dan mudah dibaca membuatnya menjadi salah satu bahasa berbasis teks yang paling mudah diakses oleh pemula. Untuk matematika, Python sangat kuat karena ekosistem perpustakaan yang luas. Perpustakaan seperti [Matplotlib](#) memungkinkan visualisasi data dan plot fungsi yang canggih, [NumPy](#) menyediakan dukungan untuk operasi pada array dan matriks besar, dan [SymPy](#) memungkinkan komputasi simbolik (aljabar). Pengenalan Python dapat dimulai di lingkungan yang lebih ramah seperti Jupyter Notebooks, yang memungkinkan pencampuran kode, teks, dan output visual dalam satu dokumen.

2.3.2. Implementasi skrip untuk simulasi konsep geometri

Platform visual sangat efektif untuk menghidupkan geometri. Sebuah implementasi skrip sederhana di Scratch dapat meminta siswa untuk memprogram sebuah *sprite* (karakter) untuk menggambar berbagai bentuk. Untuk menggambar segitiga sama sisi, siswa harus memprogram urutan: "maju 100 langkah, putar 120 derajat", dan mengulanginya tiga kali. Proses ini memberikan pemahaman yang terwujud (*embodied understanding*) tentang konsep sudut eksterior. Tugas yang lebih kompleks bisa melibatkan pembuatan program yang dapat menggambar poligon beraturan dengan jumlah sisi n , di mana n adalah input dari pengguna. Ini mengharuskan siswa untuk menggeneralisasi hubungan antara jumlah sisi dan sudut putaran ($360/n$), sebuah langkah menuju pemikiran aljabar (Brennan & Resnick, 2012).

Dalam Python, menggunakan perpustakaan seperti [turtle](#), siswa dapat mereplikasi simulasi geometri ini dalam lingkungan berbasis teks, memperkuat pemahaman mereka tentang sintaksis pemrograman. Untuk konsep yang lebih maju, [Matplotlib](#) dapat digunakan untuk memplot dan memanipulasi objek-

objek dalam sistem koordinat Kartesius. Siswa dapat menulis skrip untuk melakukan transformasi geometris, seperti translasi, rotasi, dan dilatasi, pada satu set titik dan secara visual mengamati hasilnya. Ini mengubah transformasi dari serangkaian aturan yang harus dihafal menjadi operasi dinamis yang dapat dieksplorasi.

2.3.3. Evaluasi efektivitas pemrograman visual dalam pemahaman konsep

Efektivitas pemrograman visual terletak pada kemampuannya untuk membuat konsep-konsep abstrak menjadi konkret dan dapat diamati. Ketika seorang siswa membuat kesalahan dalam memahami konsep perulangan (*loop*), dampaknya langsung terlihat: *sprite* mereka mungkin hanya menggambar satu sisi poligon alih-alih keseluruhan bentuk. Umpan balik yang instan dan visual ini menciptakan siklus belajar yang cepat: hipotesis, uji, amati hasil, perbaiki. Proses ini, yang dikenal sebagai *debugging*, secara inheren adalah proses pemecahan masalah yang melatih penalaran logis.

Studi menunjukkan bahwa keterlibatan dalam pemrograman visual dapat meningkatkan pemahaman siswa tentang konsep-konsep matematika seperti variabel, sistem koordinat, dan sudut (Grover & Pea, 2018). Namun, efektivitasnya sangat bergantung pada bagaimana aktivitas tersebut dibingkai. Jika tujuannya hanya untuk mereplikasi gambar, pembelajaran mungkin dangkal. Sebaliknya, jika guru mendorong siswa untuk memprediksi, menjelaskan, dan memodifikasi perilaku program mereka, maka platform tersebut menjadi alat yang ampuh untuk penyelidikan konseptual. Evaluasi harus fokus pada kemampuan siswa untuk mengartikulasikan hubungan antara kode yang mereka tulis dan hasil matematis yang diamati.

2.4. Panduan Praktis Integrasi STEM Digital

Mengintegrasikan STEM dan pemikiran komputasional secara efektif di kelas matematika memerlukan lebih dari sekadar pemahaman teoretis; hal ini menuntut adanya strategi praktis dan contoh-contoh konkret yang dapat diadaptasi oleh para pendidik. Panduan ini bertujuan untuk menjembatani kesenjangan antara teori dan praktik dengan menyajikan pendekatan-pendekatan yang dapat ditindaklanjuti untuk merancang pengalaman belajar interdisipliner yang memanfaatkan alat-alat digital. Kunci dari integrasi yang sukses adalah perancangan proyek yang otentik, di mana matematika dan komputasi menjadi alat yang tak terpisahkan untuk mencapai tujuan yang lebih besar dalam sains atau rekayasa.

Salah satu pendekatan praktis yang paling kuat adalah pemodelan dan simulasi fenomena sains. Banyak konsep dalam fisika, kimia, dan biologi dijelaskan oleh model-model matematis. Dengan menggunakan platform seperti Python (dengan pustaka seperti Matplotlib) atau bahkan Scratch, siswa dapat beralih dari sekadar pengguna model menjadi pencipta model. Misalnya, alih-alih hanya menggunakan rumus gerak parabola, siswa dapat menulis sebuah skrip yang mensimulasikan lintasan proyektil. Dalam

prosesnya, mereka harus menerjemahkan persamaan fisika ke dalam kode, bereksperimen dengan mengubah parameter seperti kecepatan awal dan sudut, dan secara visual mengamati bagaimana perubahan tersebut memengaruhi hasil. Proses konstruksi ini menumbuhkan pemahaman yang jauh lebih dalam tentang hubungan antar variabel.

Dalam domain rekayasa, perangkat lunak matematika dinamis seperti GeoGebra dan Desmos dapat berfungsi sebagai alat desain dan analisis yang kuat. Proyek-proyek dapat berkisar dari yang sederhana hingga yang kompleks. Contohnya, siswa dapat ditugaskan untuk merancang sebuah jembatan rangka (*truss bridge*) menggunakan alat konstruksi GeoGebra. Mereka tidak hanya harus menerapkan prinsip-prinsip geometri untuk menciptakan struktur yang stabil, tetapi mereka juga dapat menggunakan fitur-fitur platform untuk menganalisis sifat-sifat desain mereka. Dengan menambahkan *slider* untuk mengubah ketinggian atau panjang jembatan, mereka dapat menyelidiki bagaimana perubahan desain memengaruhi, misalnya, total panjang material yang dibutuhkan, sebuah masalah optimisasi sederhana.

Kunci untuk semua proyek ini adalah perancangan strategis yang disengaja. Sebuah proyek STEM digital yang efektif bukanlah sekadar aktivitas tambahan yang menyenangkan, melainkan sebuah pengalaman belajar yang dirancang secara cermat untuk mencapai tujuan kurikuler tertentu dari berbagai disiplin ilmu. Proses desain solusi digital untuk masalah STEM dapat mengikuti kerangka kerja seperti *Design Thinking*, yang melibatkan tahapan-tahapan seperti empati (memahami masalah dan pengguna), definisi (merumuskan masalah dengan jelas), ideasi (curah pendapat solusi), prototipe (membangun model solusi), dan uji coba. Pendekatan yang berpusat pada pengguna ini memastikan bahwa solusi yang dikembangkan relevan dan bermakna.

Untuk memfasilitasi proses ini, guru dapat mengembangkan "daftar periksa proyek" atau rubrik yang jelas yang menguraikan ekspektasi untuk berbagai aspek proyek. Rubrik ini tidak hanya harus menilai akurasi matematis, tetapi juga kualitas desain rekayasa, validitas model ilmiah, efektivitas penggunaan teknologi, dan keterampilan kolaborasi serta komunikasi. Menyediakan struktur semacam ini membantu siswa memahami tujuan pembelajaran yang lebih luas dan memberikan kerangka kerja untuk refleksi dan penilaian diri.

Berikut adalah tabel contoh kerangka kerja sederhana untuk merancang proyek STEM digital:

Tahapan Proyek	Pertanyaan Pemandu	Contoh Aktivitas Matematika & CT	Alat Digital Potensial
Identifikasi & Pahami	Masalah dunia nyata apa yang ingin kita selesaikan? Siapa yang terpengaruh oleh masalah ini?	Mengumpulkan dan menganalisis data awal (statistika dasar).	Spreadsheet, alat survei daring.
Curah Pendapat & Desain	Apa saja kemungkinan solusi? Kriteria apa yang harus dipenuhi oleh solusi yang baik?	Membuat sketsa dan diagram (geometri). Membandingkan pro dan kontra (logika, pengambilan keputusan).	Perangkat lunak papan tulis kolaboratif.
Modelkan & Prototipe	Bagaimana kita bisa membuat model atau simulasi dari solusi kita?	Menerjemahkan desain ke dalam kode (pemikiran algoritmik). Membuat model matematis dari sistem (aljabar, kalkulus).	Scratch, Python, GeoGebra, Desmos.
Uji & Analisis	Apakah prototipe kita bekerja sesuai harapan? Bagaimana kita bisa mengukur kinerjanya?	Mengumpulkan data dari simulasi. Menganalisis hasil dan menghitung tingkat kesalahan (statistika).	Spreadsheet, platform simulasi.
Iterasi & Komunikasi	Bagaimana kita bisa meningkatkan desain kita berdasarkan hasil pengujian? Bagaimana kita akan mempresentasikan solusi kita?	Memodifikasi algoritma atau parameter model. Membuat visualisasi data (grafik, diagram).	Perangkat lunak presentasi, alat visualisasi data.

Dengan menggunakan panduan praktis dan kerangka kerja seperti ini, pendidik dapat mulai merancang dan mengimplementasikan pembelajaran STEM digital yang koheren dan berdampak. Pendekatan ini tidak hanya membuat matematika lebih menarik dan relevan, tetapi juga secara eksplisit mengembangkan keterampilan abad ke-21 yang sangat dibutuhkan, mempersiapkan siswa untuk masa depan di mana batas-batas antar disiplin ilmu semakin kabur dan pemecahan masalah yang didukung teknologi menjadi norma.

Analogi/Contoh Kasus: Proses integrasi STEM digital dalam sebuah proyek dapat dianalogikan dengan produksi film animasi modern. Sutradara (guru) tidak hanya menyuruh animator (siswa) untuk "menggambar karakter". Sebaliknya, tim produksi harus terlebih dahulu memahami fisika gerakan (Sains) untuk membuat animasi terlihat realistis. Mereka menggunakan model-model matematis (Matematika) untuk mensimulasikan bagaimana rambut atau kain bergerak. Perangkat lunak animasi yang kompleks (Teknologi) digunakan untuk membangun dan *me-render* adegan. Seluruh proses diatur oleh prinsip-prinsip desain dan penceritaan (Rekayasa naratif). Matematika dan komputasi bukanlah subjek yang terpisah; mereka adalah alat yang tak terlihat namun esensial yang digunakan oleh para seniman untuk menciptakan produk akhir yang memukau.

2.4.1. Pemodelan Fenomena Sains : Simulasi Sains dengan Python dan Scratch

Salah satu aplikasi paling kuat dari integrasi STEM digital adalah memberdayakan siswa untuk membuat simulasi mereka sendiri. Di Scratch, siswa dapat memodelkan sistem ekologi sederhana, seperti hubungan predator-mangsa. Mereka dapat membuat *sprite* untuk rubah dan kelinci, kemudian memprogram perilaku mereka: kelinci berkembang biak, rubah memakan kelinci untuk bertahan hidup dan berkembang biak, dan keduanya mati jika tidak ada makanan. Dengan menambahkan variabel untuk mengontrol laju kelahiran dan kematian, siswa dapat menjalankan simulasi dan mengamati bagaimana populasi berfluktuasi dari waktu ke waktu, menciptakan grafik yang serupa dengan persamaan Lotka-Volterra tanpa harus menyelesaikan persamaan diferensial yang kompleks.

Dalam Python, siswa tingkat lanjut dapat melakukan pemodelan yang lebih canggih. Sebuah proyek klasik adalah simulasi gerak planet menggunakan hukum gravitasi universal Newton. Siswa perlu menerapkan konsep-konsep dari fisika (hukum Newton), matematika (vektor, kalkulus dasar untuk pembaruan posisi dan kecepatan secara bertahap), dan pemikiran komputasional (menggunakan *loop* untuk menjalankan simulasi langkah demi langkah). Dengan memvisualisasikan hasilnya menggunakan **Matplotlib**, mereka dapat secara langsung melihat bagaimana prinsip-prinsip fisika dan matematika menghasilkan orbit elips yang diamati, sebuah wawasan yang jauh lebih mendalam daripada sekadar diberi tahu rumusnya.

2.4.2. Aplikasi dalam Proyek Rekayasa (Engineering) berbantuan GeoGebra/Desmos

GeoGebra dan Desmos adalah alat yang sangat baik untuk proyek rekayasa skala kecil yang menekankan aspek desain dan optimisasi matematis. Misalnya, siswa dapat diberi tantangan untuk merancang *roller coaster* yang aman dan mendebarkan. Menggunakan Desmos, mereka dapat merancang lintasan *roller coaster* dengan menggabungkan berbagai fungsi (linear, kuadratik, polinomial, dll.). Tantangannya adalah memastikan bahwa lintasan tersebut mulus (dapat diturunkan di semua titik) dan bahwa kemiringan di titik mana pun tidak melebihi batas keamanan tertentu. Proyek ini secara alami mengintegrasikan konsep-konsep kalkulus seperti turunan (untuk kemiringan) dan kontinuitas.

Di GeoGebra, siswa dapat mengerjakan proyek desain arsitektur, seperti merancang jendela dengan bentuk geometris tertentu yang memaksimalkan luas area untuk jumlah keliling tertentu (masalah optimisasi). Mereka dapat membuat konstruksi dinamis di mana parameter desain (misalnya, radius setengah lingkaran di atas persegi panjang) dapat diubah menggunakan *slider*. GeoGebra dapat secara otomatis menghitung keliling dan luas, memungkinkan siswa untuk secara eksperimental mencari desain optimal sebelum mencoba menyelesaikannya secara analitis menggunakan kalkulus (Tamam & Dasari, 2021).

2.4.3. Strategi Desain: Solusi Digital Masalah STEM

Untuk membimbing siswa dalam mengembangkan solusi digital untuk masalah STEM, penggunaan kerangka kerja desain yang terstruktur sangatlah penting. Kerangka kerja *Understand, Define, Ideate, Prototype, Test* (UDIPT) adalah salah satu contoh yang efektif. **Understand:** Siswa melakukan riset untuk memahami konteks masalah STEM yang mereka pilih (misalnya, pemborosan energi di sekolah). **Define:** Mereka merumuskan pernyataan masalah yang spesifik dan terukur (misalnya, "Bagaimana kita bisa mengurangi konsumsi listrik untuk penerangan di ruang kelas sebesar 15%?"). **Ideate:** Siswa melakukan curah pendapat solusi digital (misalnya, membuat aplikasi yang mengingatkan guru untuk mematikan lampu, merancang simulasi untuk menunjukkan penempatan lampu yang optimal).

Tahap **Prototype** adalah di mana pemrograman dan pemodelan matematis masuk. Siswa akan membangun versi awal dari solusi digital mereka menggunakan alat seperti Scratch atau Python. **Test:** Mereka kemudian menguji prototipe mereka, baik melalui simulasi maupun dengan pengguna nyata, dan mengumpulkan data tentang kinerjanya. Proses ini bersifat iteratif; hasil dari tahap pengujian digunakan untuk kembali ke tahap sebelumnya (misalnya, memperbaiki prototipe atau bahkan mendefinisikan ulang masalah). Strategi ini tidak hanya mengajarkan keterampilan teknis, tetapi juga

menumbuhkan pola pikir inovator: kemampuan untuk mengidentifikasi masalah, merancang solusi kreatif, dan memperbaikinya berdasarkan bukti.

KESIMPULAN

Integrasi kerangka kerja STEM dan pemikiran komputasional merepresentasikan sebuah pergeseran paradigmatik yang fundamental dalam pendidikan matematika. Bab ini telah menunjukkan bahwa pendekatan ini bergerak melampaui pengajaran matematika sebagai disiplin yang terisolasi, dan menempatkannya sebagai bahasa dan alat analisis yang esensial dalam konteks pemecahan masalah dunia nyata. Fondasi STEM menyediakan "mengapa", konteks yang relevan dan otentik, sementara pemikiran komputasional menyediakan "bagaimana", seperangkat strategi berpikir sistematis untuk mengelola kompleksitas masalah-masalah tersebut.

Karakteristik utama dari pendekatan terpadu ini adalah orientasinya pada masalah, sifatnya yang interdisipliner, dan penekanannya pada proses desain dan penyelidikan. Matematika dipelajari karena dibutuhkan, bukan karena dijadwalkan, yang secara inheren meningkatkan motivasi dan pemahaman konseptual. Komponen-komponen inti dari CT, dekomposisi, pengenalan pola, abstraksi, dan desain algoritma, memberikan sebuah meta-kerangka kerja yang membuat proses berpikir matematis menjadi lebih eksplisit dan dapat diajarkan secara strategis. Siswa tidak hanya belajar untuk menemukan jawaban, tetapi juga belajar bagaimana merancang jalan menuju jawaban tersebut.

Platform komputasi visual seperti Scratch dan Python, serta perangkat lunak matematika dinamis seperti GeoGebra dan Desmos, memainkan peran krusial sebagai katalisator dalam integrasi ini. Alat-alat ini berfungsi sebagai laboratorium virtual di mana siswa dapat membangun, menguji, dan memvisualisasikan model-model matematis dan komputasional. Mereka mengubah siswa dari konsumen pasif pengetahuan menjadi produsen aktif artefak digital yang bermakna, sebuah proses yang terbukti menumbuhkan pemahaman yang lebih dalam dan lebih personal.

Panduan praktis yang disajikan dalam bab ini menggarisbawahi bahwa implementasi yang sukses bergantung pada desain pedagogis yang cermat. Proyek-proyek seperti simulasi fenomena sains dan tantangan desain rekayasa harus dirancang secara strategis untuk mencapai tujuan pembelajaran yang jelas di berbagai disiplin ilmu. Penggunaan kerangka kerja desain yang terstruktur memastikan bahwa proses kreatif siswa tetap terarah dan produktif. Pada akhirnya, sinergi antara STEM, CT, dan teknologi digital menawarkan jalan untuk mengubah kelas matematika menjadi pusat inovasi yang dinamis, di mana siswa tidak hanya belajar matematika, tetapi juga belajar menjadi pemikir, pencipta, dan pemecah masalah yang andal.

GLOSARIUM BAB

- **Abstraksi:** Salah satu komponen inti dari pemikiran komputasional, yaitu proses memfokuskan pada detail-detail yang relevan dari sebuah masalah sambil mengabaikan informasi yang tidak penting untuk menyederhanakan dan menggeneralisasi solusi.
- **Algoritma:** Serangkaian instruksi atau aturan langkah-demi-langkah yang terdefinisi dengan baik untuk menyelesaikan sebuah masalah atau melakukan sebuah komputasi.
- **Dekomposisi:** Komponen pemikiran komputasional yang melibatkan pemecahan masalah yang besar dan kompleks menjadi bagian-bagian yang lebih kecil, lebih spesifik, dan lebih mudah dikelola.
- **Konstruksionisme:** Sebuah teori belajar yang menyatakan bahwa orang belajar paling efektif ketika mereka secara aktif terlibat dalam merancang dan membangun artefak yang dapat dibagikan dan bermakna secara pribadi.
- **Pemikiran Komputasional (*Computational Thinking* - CT):** Sebuah set keterampilan pemecahan masalah yang melibatkan perumusan masalah dan solusinya dalam cara yang dapat dieksekusi oleh komputer atau agen pemroses informasi. Komponen utamanya meliputi dekomposisi, pengenalan pola, abstraksi, dan desain algoritma.
- **Pengenalan Pola:** Kemampuan untuk mengidentifikasi tren, kesamaan, atau keteraturan dalam data atau di antara masalah-masalah yang berbeda, yang memungkinkan pembuatan prediksi atau generalisasi solusi.
- **STEM (Science, Technology, Engineering, and Mathematics):** Sebuah pendekatan kurikulum dan filosofi pendidikan yang menekankan integrasi antara disiplin ilmu sains, teknologi, rekayasa, dan matematika melalui pembelajaran berbasis masalah dan proyek.

DAFTAR PUSTAKA BAB

- Brennan, K., & Resnick, M. (2012). New frameworks for studying and assessing the development of computational thinking. *Proceedings of the 2012 Annual Meeting of the American Educational Research Association, Vancouver, Canada*.
- Brennan, K., & Resnick, M. (2013). Imagining, creating, playing, sharing, reflecting: How online community supports young people as designers of interactive media. In C. Mouza & N. Lavigne (Eds.), *Approaches to studying the enablers of learning: Perspectives from the learning sciences* (pp. 297–311). SUNY Press.
- Bybee, R. W. (2010). What is STEM education? *Science*, 329(5995), 996–996. <https://doi.org/10.1126/science.1194998>
- Bybee, R. W. (2013). *The case for STEM education: Challenges and opportunities*. NSTA Press.

- Grover, S., & Pea, R. (2013). Computational thinking in K–12: A review of the state of the field. *Educational Researcher*, 42(1), 38–43. <https://doi.org/10.3102/0013189X12463051>
- Grover, S., & Pea, R. (2018). Computational thinking: A competency whose time has come. In S. Sentance, E. Barendsen, & C. Schulte (Eds.), *Computer Science Education: Perspectives on Teaching and Learning in School* (pp. 19–38). Bloomsbury Academic.
- Kelley, T. R., & Knowles, J. G. (2016). A conceptual framework for integrated STEM education. *International Journal of STEM Education*, 3(1), 11. <https://doi.org/10.1186/s40594-016-0046-z>
- Resnick, M. (2017). *Lifelong kindergarten: Cultivating creativity through projects, passion, peers, and play*. MIT Press.
- Sanders, M. (2009). STEM, STEM education, STEMmania. *The Technology Teacher*, 68(4), 20–26.
- Tamam, B., & Dasari, D. (2021). The use of Geogebra software in teaching mathematics. *Journal of Physics: Conference Series*, 1882, 012042. <https://doi.org/10.1088/1742-6596/1882/1/012042>